Design and Implementation of EPL: An Ahead-of-Time Compiled Programming Language with Compile-Time Evaluation

BY

Maksim Verevkin

Final Year project submitted in partial fulfilment of the requirements for the Degree of Bachelor of Science in Computer Science

Department of Computer Science

School of Sciences and Engineering

University of Nicosia

May 2026

Design and Implementation of EPL: An Ahead-of-Time Compiled Programming Language with Compile-Time Evaluation

BY

Maksim Verevkin

This Final Year Project has been accepted in partial fulfilment of the requirements for the Degree of Bachelor of Science in Computer Science

Project Advisor:

_____	_____	___/___/___
(Name)	(Signature)	(Date)

Examiner:

_____	_____	___/___/___
(Name)	(Signature)	(Date)

Examiner:

_____	_____	___/___/___
(Name)	(Signature)	(Date)

Department of Computer Science

School of Sciences and Engineering

University of Nicosia

Abstract

This thesis presents the design and implementation of EPL, a small ahead-of-time compiled programming language for systems programming experiments. The compiler is implemented in Rust and translates a single EPL source file through a hand-written lexer, recursive descent parser, typed tree intermediate representation, control-flow graph intermediate representation, and LLVM IR. LLVM is then used to emit a native object file that can be linked by a system C compiler.

The main contribution of the project is the `comptime` mechanism. Compile-time expressions are type checked, checked for purity, interpreted by the compiler, and replaced by constants before lower-level code generation. The evaluator supports integer arithmetic, booleans, arrays, structs, local mutation, loops, and calls to functions annotated with `@pure`.

The thesis documents the language semantics, compiler architecture, compile-time evaluator, LLVM backend, diagnostics, and testing strategy.

Acknowledgements

I would like to thank my project advisor, Professor Kjell Harald Gjermundrod, for the guidance and feedback provided during the development of this project. I would also like to thank the Department of Computer Science at the University of Nicosia for providing the academic environment in which this work was carried out.

Finally, I would like to thank my family, friends, and classmates for their support during the final year of my studies. A compiler project requires many rounds of design, implementation, debugging, and rewriting, and that process benefited from the encouragement and technical conversations that happened along the way.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Programming languages are the main interface between a programmer's ideas and a machine's execution model. A language does not only describe what a program should do; it also defines how programmers structure thought, how errors are reported, how much control is exposed, how much work can be moved from run time to compile time, and how the resulting program interacts with the operating system. Compiler construction is therefore both a theoretical and practical discipline. It combines formal language processing, type systems, intermediate representations, data layout, code generation, run-time conventions, diagnostics, testing, and engineering tradeoffs.

This Final Year Project is the design and implementation of EPL, a programming language and compiler. The language is intentionally small, but it is not merely a calculator or interpreter. It is a statically typed, expression-oriented, single-file language that compiles ahead of time to native object code through LLVM. It supports functions, local mutable variables, blocks, conditionals, loops, structs, arrays, typed raw pointers, external function declarations, integer casts, and compile-time evaluation.

The compiler is implemented in Rust and is organized as a sequence of explicit stages: lexing, parsing, typed tree lowering, compile-time evaluation, lower IR generation, CFG cleanup, LLVM IR emission, and native object generation.

The project was motivated by a common observation in modern language design: many systems languages now include some form of compile-time computation. C++ has `constexpr`, Zig has `comptime`, Rust has compile-time constants and procedural macros, and many other languages provide macro systems or specialized metaprogramming facilities. These mechanisms can remove repetitive run-time work, generate data structures during compilation, improve type safety, or help programmers express configuration in the language itself. However, a compile-time execution mechanism also forces difficult questions. Which operations are safe to execute during compilation? Can compile-time code call ordinary functions? How are mutable local variables represented? How can the compiler avoid accidentally depending on run-time state? How should the result be lowered into executable code?

EPL answers these questions conservatively. A `comptime` expression is first lowered and type checked as part of the normal program. It is then checked for purity: it may use local compile-time variables, arithmetic, loops, structs, arrays, and calls to functions marked `@pure`; it may not use string literals, pointer operations, impure function calls, surrounding run-time variables, or returns from the enclosing function. If the expression passes the check, the compiler interprets it and replaces it with a constant. This keeps the feature powerful enough to be useful while keeping the implementation small enough to be studied in a final year project.

1.1 Project Motivation

The aim of the project is educational and experimental. A production language has to solve a large number of secondary problems: package management, build systems, separate compilation, standard libraries, target-specific ABIs, tooling, long-term compatibility, editor support, optimization levels, and many more. A final year compiler project has a different goal. It should make the central ideas visible. The code should show how source text becomes tokens, how tokens become syntax trees, how names and types are resolved, how structured source constructs can be simplified into a smaller core language, how a compile-time evaluator can share the same typed representation as the run-time compiler, and how a lower representation can be mapped to LLVM.

EPL therefore follows a deliberately transparent architecture. The lexer is hand-written instead of being generated by a tool. The parser is hand-written recursive descent. The first intermediate representation, `IR_TREE`, remains tree-shaped so that it is easy to inspect and compare with the source language. The second intermediate representation is a control-flow graph with basic blocks, definitions, instructions, terminators, and block arguments. LLVM generation is explicit and uses the C API through the Rust `llvm-sys` crate. The repository also includes documentation, source examples, expected `IR_TREE` snapshots, and a small test harness.

The language is intentionally close to C in its low-level flavor. It exposes raw pointers and external function declarations, and it does not implement ownership, lifetimes, borrow checking, exceptions, garbage collection, modules, imports, or generic types. At the same time, it is more expression-oriented than C: blocks, `if`, and `loop` have result types, `return`, `break`,

and `continue` are expressions of never type, and compile-time evaluation is part of the typed expression system.

1.2 Project Contribution

The contribution of this thesis is the design and implementation of the EPL language and compiler. The project contributes the following concrete artifacts:

- a syntax and language reference for EPL;
- a hand-written lexer that recognizes identifiers, keywords, literals, punctuation, whitespace, and line comments;
- a recursive descent parser that constructs an abstract syntax tree close to the source program;
- a type system covering unit, never, booleans, signed and unsigned integers, typed pointers, opaque pointers, arrays, and structs;
- semantic analysis that resolves names, checks function signatures, validates expressions, computes aggregate layouts, and lowers source constructs into a typed `IR_TREE`;
- a purity checker and interpreter for compile-time expressions and `@pure` functions;
- a lower control-flow graph representation used for LLVM generation;
- cleanup passes that remove zero-sized operations and simplify basic control-flow patterns;
- LLVM IR generation and native object-file emission;
- a command-line interface for inspecting the AST, `IR_TREE`, lower IR, CFG, LLVM IR, and object output;
- a regression test harness using `IR_TREE` snapshots and lower IR smoke tests.

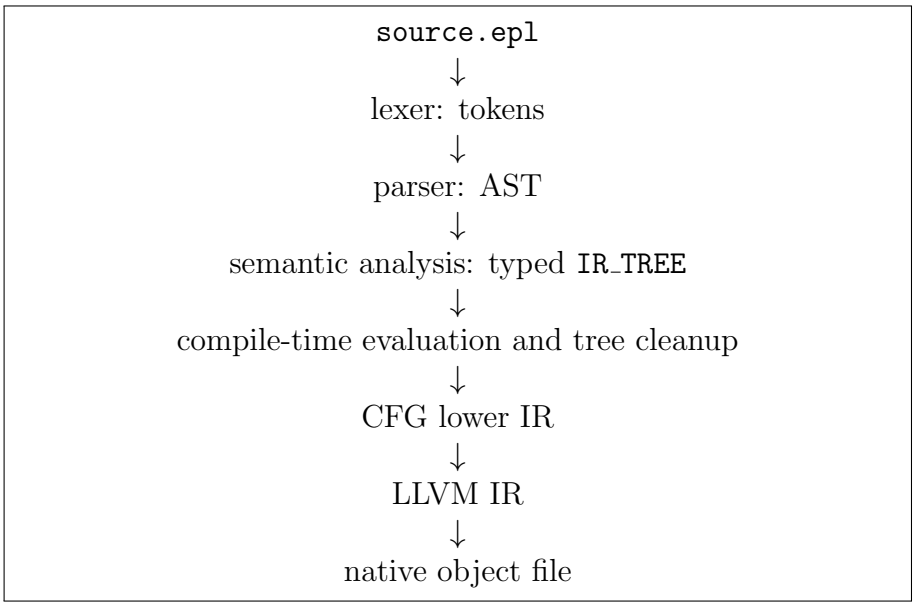

Figure 1.1: High-level pipeline of the EPL compiler

1.3 Scope and Limitations

The scope of the implementation is a working compiler for one source file at a time. It is not a complete production ecosystem. The most important current limitations are that there are no modules or imports, the C ABI is implemented only for simple external calls, there are no pointer safety checks, there are no array bounds checks, there is no name mangling or namespaces, and optimizations are intentionally limited.

These limitations are part of the design space - they show which features are essential to the central compiler pipeline. For example, a full C ABI would require target-specific calling convention work, modules and separate compilation would change the structure of name resolution. Advanced optimization would require more development time. The project focuses instead on a complete, coherent path from source to object code, as well as compile time evaluation.

1.4 Thesis Organization

The rest of this thesis is organized as follows. Chapter 2 introduces compiler concepts used throughout the thesis. Chapter 3 presents the EPL language design and semantics. Chapter 4 describes the compiler architecture and command line interface. Chapter 5 explains lexical analysis, parsing, and AST construction. Chapter 6 documents semantic analysis, type checking, `IR_TREE` lowering, and tree cleanup. Chapter 7 focuses on compile-time evaluation. Chapter 8 describes lower IR, CFG construction, and optimization passes. Chapter 9 explains LLVM generation and linking. Chapter 10 presents examples, test results, and evaluation. Chapter 10 concludes the thesis and discusses future work. The appendices include the grammar, selected examples, and representative generated output.

Chapter 2

Background

This chapter provides the background required to understand the rest of the thesis. It is not intended to be a complete introduction to compiler theory. Instead, it explains the concepts that directly influenced the design of EPL: ahead-of-time compilation, lexical analysis, parsing, static typing, intermediate representations, control-flow graphs, SSA-like values, compile-time evaluation, foreign function interfaces, and LLVM.

2.1 Compiled Languages

A compiler translates a program from one language into another language. In a systems programming context, the source language is usually a human-readable programming language and the output is usually machine code, assembly, object code, bytecode, or another lower-level representation. Classic compiler texts describe this as a sequence of front-end, middle-end, and back-end phases [1, 2]. The front end understands source syntax and semantics. The middle end performs analysis and transformations. The back end emits code for the target execution environment.

An ahead-of-time compiler performs this translation before the program is run. This differs from an interpreter, which executes a program directly, and from a just-in-time compiler, which compiles parts of the program while it is already running. Ahead-of-time compilation is suitable for EPL because the project targets native object output and C-compatible linking. The compiler performs all source analysis before execution, emits an object file, and leaves final linking to a system C compiler.

Ahead-of-time compilation has several advantages for a systems language. Errors can be detected early, generated code can be inspected, external symbols can be linked by existing toolchains, and the final executable does not need to include an interpreter. It also has costs. Compilation must know enough about all referenced functions and types up front, compile-time execution must be restricted to information available at compilation, and the implementation must make target-specific decisions about data layout and calling conventions.

2.2 Lexical Analysis

Lexical analysis is the first stage of most compilers. The lexer reads raw source text and produces a sequence of tokens. Tokens are the meaningful units of the surface language: identifiers, keywords, literals, operators, delimiters, and punctuation. Whitespace and comments are usually removed at this stage unless they are semantically important.

The EPL lexer is hand-written. It recognizes identifiers, reserved keywords, number literals, string literals, punctuation, whitespace, and comments beginning with the `#` character. It also attaches spans to tokens. A span is the start and end byte offset of a source fragment. Later stages use

spans to report diagnostics at the correct line and with a caret underline.

Using a hand-written lexer has an educational advantage: the exact rules are visible in ordinary Rust code. It also matches the scale of the language. EPL does not require indentation-sensitive lexing, contextual lexing, Unicode identifier normalization, or complex preprocessor behavior. A compact iterator over tokens is sufficient.

2.3 Parsing and Abstract Syntax Trees

Parsing turns tokens into a tree that reflects the grammar of the language. The abstract syntax tree, or AST, records the structure of a source program without yet resolving every semantic question. For example, in the AST a type name is still an identifier string. A variable use is still a name. A binary expression has a left operand, an operator, and a right operand, but it does not yet know whether the operands are integers or booleans.

EPL uses recursive descent parsing. In recursive descent, grammar rules are implemented as functions that call one another. This style is straightforward for languages whose grammar can be parsed with a small amount of lookahead. It is also easy to connect expression precedence to function structure: assignment calls logical-or parsing, logical-or calls logical-and parsing, comparison calls range parsing, range calls addition, addition calls multiplication, and so on. The approach is common in educational compilers because it keeps parsing logic close to the grammar [3].

The EPL AST deliberately stays close to source syntax. It stores annotations, function declarations, struct definitions, block expressions, array and struct initializers, control-flow expressions, casts, calls, assignments, and all source-level operators. Most semantic checks are delayed until typed

lowering. This separation keeps parsing focused on syntax and keeps type checking focused on meaning.

2.4 Static Typing

A statically typed language assigns a type to each expression before the program is executed. Static typing helps catch errors early, documents programmer intent, and provides information needed for code generation. In a compiler targeting native code, type information is also needed to choose instruction widths, compute memory layouts, decide calling conventions, and validate operations.

EPL is statically typed and intentionally avoids implicit numeric conversions. Signed and unsigned integer types of the same width are distinct in the typed tree. This decision prevents surprising conversions and makes type checking simple: arithmetic operands must have the same integer type, casts must be explicit, and a function call argument must match the parameter type. The lower IR later collapses signed and unsigned integers of the same width into the same storage type while preserving signedness on operations that need it.

The language also includes two special types. The `unit` type is a zero-sized type used for expressions with no meaningful value, similar to `void` in C but represented as an ordinary value type in the compiler. The never type, written `!`, is used for expressions that do not complete normally, such as `return`, `break`, and calls to external functions such as `exit`. Treating never as a type simplifies the typing of expression-oriented control flow.

2.5 Intermediate Representations

An intermediate representation, or IR, is a compiler-internal representation of a program. Compilers usually use more than one IR because different stages need different shapes of information [4]. A source-like tree is convenient for type checking and diagnostics, while a control-flow graph is more suitable for code generation and optimization.

EPL uses two main intermediate representations. The first is `IR_TREE`, a typed expression tree. It is source-like enough to be readable but semantic enough that every expression has a type, variables and functions are represented by generated IDs, aggregate layouts have been computed, and several source constructs have been lowered into simpler forms. The second is lower IR, a control-flow graph with basic blocks, instructions, terminators, definitions, and block arguments. Lower IR is closer to LLVM and makes branches, returns, loads, stores, calls, pointer offsets, casts, and arithmetic explicit.

Representation	Main purpose	Important properties
AST	Preserve source structure after parsing	Names and type names are still unresolved; spans are kept for diagnostics.
`IR_TREE`	Semantic analysis and compile-time evaluation	Every expression is typed; variables/functions/loops use IDs; source constructs are simplified.
Lower IR	Code generation and CFG cleanup	Basic blocks, instructions, terminators, block arguments, explicit memory operations.
LLVM IR	Target-independent backend representation	Verified by LLVM and emitted as native object code.

Table 2.1: Intermediate representations used by the EPL compiler

2.6 Control-Flow Graphs and SSA-Like Values

A control-flow graph represents a function as basic blocks connected by terminators. A basic block is a sequence of instructions with one entry and one terminator. Terminators include jumps, conditional jumps, returns, and unreachable markers. CFGs make the possible execution paths through a function explicit.

Static Single Assignment form, or SSA, represents each computed value as being assigned exactly once. SSA is widely used because it simplifies many analyses and optimizations [5]. EPL does not implement a full source-variable SSA construction pass. Local mutable variables are represented as stack allocation slots in lower IR, and loads and stores are explicit.

However, computed lower IR values are immutable definitions, and basic block arguments serve a similar role to phi nodes at join points. This is a common way to model SSA-like control flow: predecessor blocks pass values to successor block arguments, and LLVM generation turns those block arguments into LLVM phi nodes.

2.7 Compile-Time Evaluation

Compile-time evaluation means executing some part of a program while compiling it, then using the result in the generated program. Partial evaluation and metaprogramming have a long history in programming language research [6]. Modern systems languages use compile-time computation for constants, generic specialization, code generation, configuration, and data precomputation.

Compile-time evaluation is attractive because it moves work out of the final executable. It can also let programmers write ordinary code to compute constants instead of using external generators. The difficulty is that not all operations make sense at compile time. Reading a run-time variable, dereferencing a raw pointer, calling an impure external function, performing I/O, or depending on target process state would make the compiler's behavior unsafe or non-deterministic.

EPL handles this by defining a pure compile-time subset. The subset is not a separate language. It is the same expression language after type checking, but with a purity check before interpretation. This design avoids building a second parser or evaluator for a separate macro language.

2.8 LLVM

LLVM is a compiler infrastructure project that provides a typed intermediate representation, optimization passes, target backends, object emission, and tools [7, 8]. Many language implementations use LLVM to avoid writing a complete machine-code backend. A compiler can lower its own IR to LLVM IR, verify the LLVM module, and ask LLVM to emit assembly or object code for a target platform.

EPL uses LLVM through the Rust `llvm-sys` crate. The compiler declares all functions first, emits each function body into LLVM basic blocks, creates allocas for local storage, emits loads and stores, maps lower IR block arguments to LLVM phi nodes, emits integer arithmetic and comparisons, and finally asks LLVM to emit a native object file. The object file is then linked with a system C compiler so that external functions such as `printf` or `exit` can be resolved from libc.

Chapter 3

The EPL Language

This chapter describes the EPL language from the programmer's point of view. The implementation is intentionally small, so the language can be explained without hiding large subsystems. Even so, EPL includes enough features to compile useful examples: functions, external declarations, local variables, expression blocks, conditionals, loops, structs, arrays, raw pointers, casts, and compile-time evaluation.

3.1 Design Goals

The first design goal is simplicity. EPL is a single-file language. There are no modules, imports, packages, namespaces, generics, traits, classes, closures, or overloads. Function names are global and are emitted as written. Struct names live in a separate global type namespace. This keeps name resolution and code generation understandable.

The second design goal is explicitness. Numeric conversions are not implicit. Pointer operations are written explicitly. External functions are declared explicitly. A function's parameter types are always written in

the source. The compiler performs local type inference for variables and literals, but it does not try to infer function signatures or perform overload resolution.

The third design goal is a clear route to native code. The language has C-like integers, raw pointers, and external declarations because these map naturally to LLVM and to platform linkers. The compiler does not contain a full linker. It emits an object file and expects a system C compiler to produce the final executable.

The fourth design goal is compile-time execution. EPL should be able to evaluate selected code during compilation without turning the language into a separate macro system. A `comptime` expression is ordinary EPL code with extra purity restrictions.

3.2 Program Structure

An EPL source file is a sequence of top-level items. The current item kinds are function definitions, function declarations, and struct definitions. Functions and types are stored in separate namespaces. Function declarations are collected before function bodies are checked, so functions may call functions that are declared later in the file and may be recursive. Struct definitions are processed in source order, so a struct field may only refer to a type already known at the point of the struct definition.

A function definition has a name, parameters, an optional return type, and a body. A function declaration has the same signature form but ends with a semicolon instead of a body. Function declarations are how EPL names external functions. Listing 3.1 shows the standard hello-world example.

Listing 3.1: Hello world in EPL

```
fn main() -> i32 {
  printf("hello world\n");
  0
}

fn printf(fmt: *i8, ...);
```

The compiler enforces the entry point signature `fn main() -> i32`. The language allows external variadic function declarations, as shown with `printf`, but it does not allow defining variadic functions in EPL. This is a deliberate implementation boundary: calling simple variadic C functions is useful for examples, while implementing and lowering variadic EPL function bodies would add ABI complexity.

3.3 Types and Values

Every expression in EPL has a statically known type after semantic analysis. The built-in types are:

- `unit`, the zero-sized type for expressions that produce no meaningful value;
- `bool`, with values `true` and `false`;
- signed integers `i8`, `i32`, and `i64`;
- unsigned integers `u8`, `u32`, and `u64`;
- `ptr`, an opaque pointer type;
- `!`, the never type for expressions that do not complete normally.

Composite types are typed pointers, arrays, and user-defined structs. A typed pointer is written `*T`. An array type is written `[T; N]`, where `N` is currently a number literal. Structs are declared by name and store their fields by value.

Integer literals may have suffixes such as `10u32` or `0i64`. If a literal has no suffix and the surrounding context expects an integer type, the literal uses that expected type. Otherwise, unsuffixed integer literals default to `i32`. String literals have type `*i8` and are intended for simple C-style external calls.

The literal `undefined` produces an undefined value of an expected type. It therefore requires context. For example, `let x: i32 = undefined;` is valid, while `let x = undefined;` is not, because the compiler cannot infer the type from the value.

3.4 Variables and Scope

Local variables are introduced with `let`. Variables are mutable. A `let` declaration can include an explicit type, an initializer, or both:

Listing 3.2: Variable declaration forms

```
let a = 10;
let b: u64 = 20;
let c: [i32; 4];
```

Blocks introduce lexical scopes. Name lookup starts in the current scope and then searches outer scopes. A later declaration may shadow an earlier binding in the same or an inner scope. Function arguments behave as mutable local variables inside the function body because lowering turns each argument into an ordinary local variable initialized from an argument expression. This choice simplifies assignment to arguments and keeps all local storage in one model.

Reading a local value before assigning to it is undefined. The compiler reserves storage for declarations without initializers, but it does not imple-

ment a definite-assignment analysis. This is another example of a conscious scope decision: the language can express low-level uninitialized storage, but safety checks around it are not yet implemented.

3.5 Expressions and Blocks

EPL is expression-oriented. A block evaluates its statements in order. If it ends with a final expression, the block's type is the type of that expression. If there is no final expression, the block has type `unit`. Expression statements discard their value.

The following function uses a block as the result of an `if` expression:

Listing 3.3: Expression-oriented conditional

```
fn abs_i32(x: i32) -> i32 {
  if x < 0 {
    -x
  } else {
    x
  }
}
```

The compiler checks that the branches of an `if` expression have compatible types. An `if` without `else` is only accepted when a `unit` result is expected. This prevents accidentally using a non-exhaustive conditional where a value is required.

3.6 Control Flow

The language supports `return`, `break`, `continue`, `if`, `loop`, `while`, and `for`. The `return` expression exits the current function and has type `!`. `break` and `continue` are valid only inside loops and also have type `!`. A

`loop` expression can produce a value if it exits through `break value`. A `while` expression has type `unit`.

`for` loops currently support half-open integer ranges of the form `start..end`. The start and end expressions are evaluated once before the loop. The compiler lowers a `for` loop into a block containing hidden counter and target variables plus a normal `loop`. The visible loop variable is assigned a per-iteration copy, so assigning to it does not modify the hidden counter.

Listing 3.4: Loop forms used in EPL

```
fn sum_to(n: u32) -> u32 {
  let total: u32 = 0;
  for i in 0u32..n {
    total += i;
  }
  total
}
```

The lowering strategy is important because it keeps the rest of the compiler small. After semantic analysis, there is no special lower IR operation for `for`. There is only block, store, comparison, arithmetic, and loop.

3.7 Operators and Casts

Arithmetic operators are supported for integer operands only. Both operands must have the same integer type. Supported operations are addition, subtraction, multiplication, division, remainder, and unary negation for signed integers. Compound assignment operators modify a place expression in place.

Comparison operators return `bool`. Integers support all comparison operators. Booleans support equality and inequality. Pointer comparisons

are not currently implemented.

Logical `&&` and `||` require boolean operands and short-circuit. They are lowered to conditional expressions. For example, `a || b` returns `true` when `a` is true and otherwise evaluates `b`. This is more explicit than carrying logical operators through every compiler stage.

The supported casts are integer-to-integer and pointer-to-pointer. Narrowing integer casts truncate. Widening integer casts sign-extend signed source values and zero-extend unsigned source values. Pointer-to-pointer casts preserve the pointer value. Pointer-to-integer and integer-to-pointer casts are not supported.

3.8 Structs and Arrays

Struct definitions list named fields. Field names must be unique within a struct. Struct values are passed, returned, assigned, and stored by value. Initializers must provide every field exactly once, although the initializer field order does not need to match the declaration order.

Listing 3.5: Struct declaration and initialization

```
struct Vec2 {
  x: i32,
  y: i32,
}

fn main() -> i32 {
  let vec = Vec2 .{ x: 10, y: 20 };
  0
}
```

Arrays are fixed-size values. Array indexing requires a `u64` index. Indexing does not perform bounds checks. Like structs, arrays are stored by value. The compiler computes array layout from the element type layout

and the literal length in the type.

3.9 Pointers and External Functions

EPL exposes raw typed pointers. The expression `&place` takes the address of a place. The expression `ptr.*` dereferences a typed pointer. The opaque `ptr` type cannot be dereferenced because it has no pointee type. The language does not implement lifetime checking, borrow checking, null checking, pointer arithmetic, alias analysis, or bounds checking.

Pointers are needed both as a systems programming feature and as a bridge to external C functions. The examples use declarations such as:

Listing 3.6: External declarations

```
fn printf(fmt: *i8, ...);
fn exit(code: i32) -> !;
```

These declarations are emitted as LLVM function declarations. The final system link step resolves the symbols. The C ABI support should be treated as limited: simple integer and pointer calls work for the examples, but by-value aggregate ABI details, exact C `void` behavior, platform-specific variadic requirements, and complex signatures are not fully covered.

3.10 Compile-Time Evaluation

The `comptime` keyword asks the compiler to evaluate an expression during compilation. The result is inserted into the typed tree as a constant before lower IR generation. Listing 3.7 shows a simplified example derived from the project examples.

Listing 3.7: Computing values at compile time

```
let primes = comptime {
  let lut: [i32; 20];
  let ready = 0;
  let next = 3;
  while ready < 20 {
    while !is_prime(next) {
      next += 2;
    }
    lut[ready as u64] = next;
    ready += 1;
    next += 2;
  }
  lut
};
```

A compile-time expression may use literals except strings, arithmetic, comparison, boolean operations, integer casts, blocks, conditionals, loops, local variables, local mutation, arrays, structs, `break`, `continue`, and calls to `@pure` functions. It may not access surrounding run-time variables, use strings, take addresses, dereference pointers, call impure functions, or return from the enclosing function.

This design is the main language-level feature that distinguishes EPL from a minimal C-like compiler. It allows non-trivial computation during compilation without adding a separate macro language.

Chapter 4

Compiler Architecture

This chapter describes the architecture of the EPL compiler as implemented in the repository. The compiler is a Rust workspace with two crates. The main crate, `epl`, contains the compiler and command-line interface. The helper crate, `epl_test`, contains the snapshot and smoke-test harness.

4.1 Repository Structure

The source tree is intentionally small. The main compiler modules are located in `src/`. Documentation is located in `doc/`. Example programs and their expected `IR_TREE` snapshots are located in `examples/`. Additional regression tests are located in `tests/`.

Path	Purpose
`src/lex.rs`	Lexer, tokens, spans, and lexical errors.
`src/ast.rs`	AST definitions and recursive descent parser.
`src/ir_tree.rs`	Typed tree IR data structures and module construction.
`src/ir_tree/lower_ast.rs`	Semantic lowering from AST to `IR_TREE`.
`src/ir_tree/checkers.rs`	Main signature, purity, and compile-time checks.
`src/ir_tree/evaluator.rs`	Compile-time evaluator.
`src/ir.rs`	Lower IR data structures.
`src/ir/lower_ir_tree.rs`	Lowering from `IR_TREE` to CFG IR.
`src/ir/opt/`	Lower IR cleanup passes.
`src/llvm.rs`	LLVM module construction and object emission.
`src/diagnostics.rs`	Shared diagnostic printing.
`epl_test/src/main.rs`	Test harness over examples and tests.

Table 4.1: Main implementation files in the EPL repository

4.2 Command-Line Interface

The compiler executable accepts a command and a source file. The command chooses which pipeline stage is printed or emitted. This is useful both for users and for compiler development because each representation can be inspected independently.

Listing 4.1: Compiler command-line modes

```
epl ast <file>
epl ir_tree <file>
epl ir <file>
epl cfg <file>
epl llvm-ir <file>
epl llvm-obj <file>
```

`ast` prints the parsed AST. `ir_tree` prints the typed tree. `ir` prints the lower IR debug representation. `cfg` prints a Graphviz DOT graph of the lower IR control-flow graph. `llvm-ir` prints the generated LLVM module. `llvm-obj` asks LLVM to emit a native object file named `a.out.o`.

The commands are not separate compilers. They call the same staged functions: parse to AST, lower to IR_TREE, lower to IR, build an LLVM module, and optionally emit an object file. If an earlier stage fails, diagnostics are printed and the process exits.

4.3 Pipeline Overview

The full compiler pipeline can be summarized as:

Listing 4.2: Compiler pipeline

```
source text
  -> lexical tokens
  -> AST
  -> typed IR_TREE
  -> evaluated comptime constants
  -> lower CFG IR
  -> cleaned CFG IR
  -> LLVM IR
  -> native object file
```

Each stage has a deliberately narrow responsibility. The lexer does not know about types. The parser does not resolve variable names. IR_TREE lowering performs most semantic checks. The compile-time evaluator operates on typed IR_TREE expressions rather than the AST. Lower IR generation makes control flow and memory operations explicit. LLVM generation maps this lower IR to the LLVM C API.

This separation is important for debugging. When an example does not compile, the developer can inspect the AST to see whether parsing was correct, inspect the IR_TREE snapshot to see whether semantic lowering was correct, inspect the lower IR to understand CFG construction, or inspect LLVM IR to debug backend generation.

4.4 Entity Identifiers

Source names are convenient for programmers but inconvenient for compiler internals. A local variable can shadow another local variable with the same name. A loop may contain nested loops. A function declaration may be referenced before its body appears. The compiler therefore uses generated entity IDs for functions, variables, and loops after semantic lowering.

The `make_entity_id!` macro creates small strongly typed wrappers around non-zero integer IDs. The typed wrappers prevent accidentally mixing a function ID with a variable ID or loop ID. They also provide compact debug formatting, such as `fn_1`, `var_3`, and `loop_2`. This is a small implementation detail, but it improves the readability of generated snapshots.

4.5 Type System Context

The type system stores built-in types, struct definitions, and interned type IDs. Struct and array types need layout information. The compiler computes each struct's field offsets, size, and alignment when the struct definition is processed. Array layout is computed from the element layout and length. Pointer layout uses the target pointer size stored in the type system context. The current implementation uses an 8-byte pointer size, matching common 64-bit targets.

The type system uses a distinction between a `Type` value and a `TypeId`. Many types can be stored directly in a compact enum, but typed pointers and arrays need stable references to pointee or element types. `TypeId` provides this stable reference. For example, `*T` stores the ID of `T`, and `[T;`

`N]` stores the ID of `T` plus the array length.

4.6 Diagnostics

The compiler stages expose different error types, but they all adapt to a shared diagnostic interface. The diagnostic printer displays the message, file path, line number, source line, and a caret underline for the span. This is why the lexer and parser retain byte offsets and why later semantic errors carry spans from AST nodes.

Good diagnostics matter even in a small compiler. A type error without a source location is difficult to fix. A parser error that does not say what token was expected is frustrating. The current diagnostic system is simple, but it already captures the most important user-facing behavior: an error points to the source fragment that caused it.

4.7 Documentation and Examples

The repository includes four documentation files: syntax reference, language reference, compiler architecture, and LLVM setup. These files serve two purposes. First, they document the user-visible language rules. Second, they provide a specification against which the implementation and thesis can be checked.

The examples cover hello world, recursion, iterative loops, arrays, structs, pointers, external `exit`, `else if`, and compile-time prime generation. Each example has a neighboring `IR_TREE` snapshot. The snapshots are not only tests. They are executable documentation of lowering decisions, such as how `for` loops become hidden counter variables or how a `comptime` block

becomes a constant array.

4.8 Build Environment

The Rust workspace uses edition 2024 and depends on `llvm-sys` version `221.0.0`. The project includes a Cargo configuration file that points `LLVM_SYS_221_PREFIX` to an LLVM 22.1.2 installation. The compiler can be built with:

Listing 4.3: Building the compiler

```
cargo build
```

The test harness can then be run with:

Listing 4.4: Running the EPL test harness

```
cargo run -p epl_test ./target/debug/epl examples tests
```

The command in Listing above is also used in Chapter 10 to report the verification status of the implementation.

Chapter 5

Front End Implementation

The front end of a compiler converts source text into a structured form that can be analyzed. In EPL, the front end consists of the lexer and parser. The lexer produces tokens with spans. The parser consumes those tokens and constructs an AST. This chapter explains the implementation choices and how the source language is represented before semantic analysis.

5.1 Lexer Design

The lexer is implemented in `src/lex.rs`. It is a Rust iterator over `Result<(Span, Token), Error>`. Each successful item contains a span and a token. Each failed item contains a lexical error with the span where the problem was found.

The lexer stores two pieces of state: the current byte offset and a reference to the source string. On each iteration it skips whitespace and comments, then tries to recognize the next token. Comments start with `#` and continue until the end of the line. This simple rule avoids interaction with nested block comments or preprocessor-like behavior.

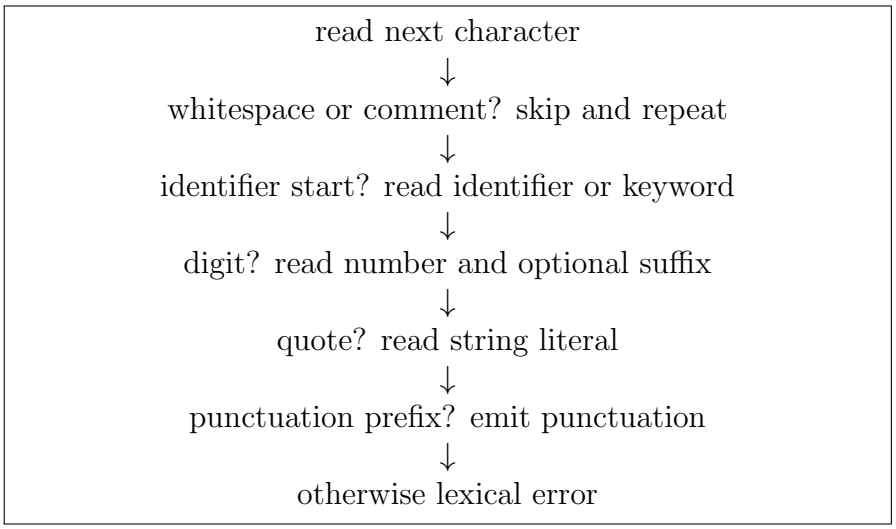

Figure 5.1: Control flow of the EPL lexer

5.2 Tokens

The token enum has four main variants: keyword, identifier, literal, and punctuation. Keywords are stored in a fixed keyword enum. Identifiers store their source string. Literals currently include number literals and string literals. Punctuation includes operators and delimiters such as `+`, `==`, `->`, `...`, the dot-left-brace token, parentheses, braces, and brackets.

The lexer checks punctuation by trying entries in a punctuation table ordered from longer strings to shorter strings. This matters because `...` must be recognized before `..`, `==` before `=`, and `->` before `-`. The implementation keeps this priority visible in one table.

Number literals are parsed as `i128` first. This lets the lexer accept a large neutral representation before semantic analysis chooses the target integer type. If the number text does not fit into `i128`, the lexer reports `NumberLiteralTooLarge`. A number may be followed by an identifier suffix without whitespace, such as `42u64`. The suffix is stored with its own span

so semantic analysis can report unknown suffixes precisely.

String literals support a small set of escape sequences for newline, carriage return, tab, and a literal backslash. Unknown escape sequences and unclosed strings are lexical errors. The language does not currently include character literals or raw strings.

5.3 Spans

A span is a half-open byte range `start..end` into the source string. Spans are attached to tokens, AST identifiers, type nodes, expression nodes, and many semantic errors. The `Span::join` method creates a larger span covering two smaller spans. This is useful when an expression spans multiple tokens, for example when a binary expression spans from the start of its left operand to the end of its right operand.

The diagnostic printer uses spans to compute the line number and underline. This is why even small AST nodes, such as identifiers and punctuation-driven constructs, preserve source locations. A compiler that drops source locations in the parser can still compile code, but it cannot explain errors well.

5.4 Parser Design

The parser is implemented in `src/ast.rs`. It owns the lexer and a small lookahead queue. The parser exposes a single public entry point: `Parser::new(src).parse()`. The result is an `Ast` containing a vector of top-level items.

The parser is recursive descent. Each grammar category is represented by

a method. For example, `next_function` parses a function item, `next_type` parses a type, `next_block_expr` parses a block, and expression precedence is implemented by methods such as `next_or_expr`, `next_and_expr`, `next_comp_expr`, `next_additive_expr`, and `next_multiplicative_expr`.

The parser performs syntax validation but not full semantic validation. It can reject duplicate annotations on an item, reject `let name;` without type or initializer, require variadic `...` to be the final function parameter, and report unexpected tokens. It does not check whether a type name exists, whether a variable is in scope, whether a function call has the correct argument types, or whether an arithmetic operator is applied to integers. Those checks belong to semantic lowering.

5.5 Top-Level Items

An EPL source file is parsed as zero or more top-level items. Each item may have annotations. The parser stores annotations in a `BTreeSet` keyed by annotation name, which makes duplicate detection simple.

Function parsing handles both definitions and declarations. The parser reads the `fn` keyword, the function name, parameter list, optional return type, and then either a block body or a semicolon. A semicolon means that the function is a declaration. Variadic functions are represented by a boolean flag on the AST function.

Struct parsing reads a name and a comma-delimited list of fields. Field types are parsed as AST type nodes, not resolved to semantic types yet. This is important because struct processing later uses the global type namespace and computes layout.

5.6 Expression Parsing

The expression grammar is layered by precedence. Assignment has low precedence, logical operators are above assignment, comparison is above logical operators, range is above comparison, addition is above range, multiplication is above addition, casts are above multiplication, unary operators are above casts, and field/index/dereference access is above unary operations.

The parser treats several constructs as expression forms: `return`, `break`, `continue`, `comptime`, blocks, `if`, `loop`, `while`, `for`, array initializers, struct initializers, calls, casts, unary operations, binary operations, assignments, identifiers, and literals.

Blocks require a small parsing rule because EPL distinguishes statements from the final expression. A block may contain semicolons, `let` statements, expression statements, and an optional final expression. If the next expression is followed by a closing brace, it becomes the final expression. If it is followed by a semicolon, it becomes an expression statement.

5.7 AST Shape

The AST is deliberately close to source syntax. The `Expr` enum includes variants for `While`, `For`, `Range`, `LogicalAnd` and `LogicalOr` as binary operations, `AsCast`, `Comptime`, and other source-level forms. Later stages lower these into a smaller semantic core. For example, `while` does not survive as a distinct lower construct; it becomes a `loop` containing an `if`.

Keeping the AST close to source syntax has two advantages. First, it makes AST debug output useful when diagnosing parser behavior. Second, it avoids mixing syntax parsing with language design decisions that are

easier to make after types are known.

5.8 Parser Error Handling

Parser errors contain an optional span and an error kind. Unexpected token errors record both the expected description and the token that was actually found. Lexical errors are wrapped so the parser can expose one error type to the rest of the compiler.

The parser stops at the first error. It does not implement error recovery or multi-error reporting. For a teaching compiler this is acceptable, but it is a clear area for future improvement. Production compilers often try to recover after syntax errors so they can report several problems in one run.

Chapter 6

Semantic Analysis and IR_TREE

Semantic analysis is the stage where the compiler decides what the program means. In EPL, semantic analysis is implemented primarily by lowering the AST into `IR_TREE`. The resulting representation is still expression-oriented, but it is no longer source syntax. Every expression has a type, every local variable and function has an ID, struct and array layouts are known, and several source constructs have been rewritten into simpler core forms.

6.1 Purpose of IR_TREE

`IR_TREE` is the main semantic representation of the compiler. It exists between the AST and the lower CFG IR. It is needed because the AST is too source-like for code generation, while the lower CFG IR is too low-level for convenient semantic checking and compile-time evaluation.

The key properties of `IR_TREE` are:

- each expression carries a static type;

- local variables are represented by `VariableId`, not by source strings;
- functions are represented by `FunctionId`;
- loops are represented by `LoopId`, which connects `break` and `continue` to the correct loop;
- places are represented separately from values;
- arrays, structs, field access, and pointer dereference have explicit typed forms;
- source conveniences such as `while`, `for`, `&&`, and `||` are lowered to simpler expression forms.

Because `IR_TREE` is readable, it is also used by the test harness as a snapshot format. The snapshots are not a formal stable API, but they are very useful for regression testing and for explaining the compiler in this thesis.

6.2 Module Construction

`Module::from_ast` constructs a typed module from the AST. It performs several passes:

1. create a new type system context and seed the global type namespace with built-in types;
2. collect struct definitions and compute their layouts;
3. collect function declarations, including signatures, return types, variadic flags, and `@pure` annotations;
4. lower each function body using the completed type and function namespaces;
5. run semantic checkers;
6. run basic tree optimizations;
7. evaluate every remaining `comptime` expression and replace it with a constant.

This ordering is important. Function declarations must be known before function bodies are lowered so a function can call another function that appears later in the file. Struct definitions are collected before function bodies, but their fields are processed in source order. This means a struct definition can refer only to types already known at that point.

6.3 Type Resolution

AST types are resolved by looking up identifier names in the type namespace. Built-in types are inserted before source items are processed. Struct definitions add new entries to the namespace. Pointer and array types are constructed recursively.

Struct layout follows a simple alignment algorithm. For each field, the current size is rounded up to the field's alignment, the field offset is recorded, and the field size is added. The final struct size is rounded up to the maximum field alignment. Arrays use the element layout and multiply by the length. Unit and never have zero size. Booleans and `i8`/`u8` have one-byte layout. `i32`/`u32` have four-byte layout, and `i64`/`u64` have eight-byte layout.

6.4 Name Resolution and Scope

Function body lowering uses a `FunctionLoweringCtx`. The context stores the function declaration, function namespace, function table, type system, type namespace, and current lexical scope. A scope contains a map from source variable names to `VariableId` and type pairs. It also stores optional loop context for resolving `break` and `continue`.

When entering a block, the compiler pushes a new scope. When leaving the block, it pops the scope. Lookup searches the current scope and then recursively searches parent scopes. This implements lexical scoping and shadowing.

Function parameters are lowered into local variables at the beginning of the function body. Each parameter receives a variable ID and a declaration in the outer function block. The body begins with stores from `Argument(n)` expressions into those locals. This design makes parameters behave like mutable locals and avoids a second storage model for arguments.

6.5 Expected Types and Local Inference

EPL uses local, context-driven type inference. The compiler does not infer function signatures, but it can infer the type of a local variable from its initializer, infer unsuffixed integer literal types from the expected type, and propagate expected types into several expression forms.

Expected types are passed into lowering for:

- typed `let` initializers;
- assignment right-hand sides;
- fixed function call arguments;
- function return expressions;
- block final expressions;
- `if` branches;
- unnamed struct initializers;
- array initializers;
- `undefined`;
- unsuffixed integer literals.

This approach gives useful inference while keeping the implementation simple. For example, `let x: u64 = 0;` works because the expected type `u64` flows into the literal `0`. Without that context, the same literal would default to `i32`.

6.6 Places and Values

The compiler distinguishes places from values. A place is something that can be assigned to or addressed: a local variable, a field of a place, an array element of a place, or a dereferenced pointer. A value is the result of evaluating an expression. Source syntax does not always expose this distinction, so semantic lowering derives it.

For example, parsing `x.y` produces a field-access AST expression. During semantic lowering it can become either a value field access or, if used on the left-hand side of assignment, a field place. The `Expr::into_place` method converts suitable load, field, and array element expressions into a `Place`. If conversion fails, the compiler reports "expected a place expression."

This distinction is essential for assignment, compound assignment, address-of, and pointer dereference. It also makes lower IR generation easier because places can be lowered to pointers.

6.7 Lowered Source Constructs

Several source constructs are rewritten during `IR_TREE` lowering:

- a `while` expression becomes a `loop` containing a conditional break;
- a `for` expression over a range becomes a block with hidden counter and target variables plus a normal `loop`;

- `a || b` becomes a conditional expression that skips `b` when `a` is true;
- `a && b` becomes a conditional expression that skips `b` when `a` is false;
- unary negation becomes subtraction from zero;
- function arguments become local variables initialized from argument expressions.

Lowering source constructs early reduces the number of forms later stages must understand. Lower IR does not need a special `while` instruction, a special `for` instruction, or a short-circuit boolean instruction. It only needs blocks, loops, conditionals, arithmetic, comparisons, loads, stores, and calls.

6.8 Semantic Checkers

After function bodies are lowered, the compiler runs semantic checkers. The first checker verifies the `main` ABI. If a function named `main` exists, it must have no arguments, must not be variadic, and must return `i32`.

The second checker validates `comptime` expressions. It walks each function body and checks the inner expression of each `comptime`. It rejects operations that would depend on run-time state or unsafe effects.

The third checker validates `@pure` functions. A pure function must have a body, may call only other pure functions, and may not use operations that are not allowed in pure code. This check is needed because compile-time expressions may call pure functions.

6.9 Tree Optimization

The `IR_TREE` optimizer is intentionally small and local. It performs transformations that simplify generated trees without requiring global data-flow analysis:

- `&ptr.*` becomes `ptr`;
- `(&place).*` becomes `place`;
- empty blocks and single-expression blocks with no local variables are collapsed;
- code after the first expression of never type in a block is removed;
- pure unused expressions before the final expression are removed;
- redundant trailing unit constants are removed.

Listing 6.1 shows an example. The source calls `external()`, returns, and then calls `external()` again. The second call is unreachable and is removed from the `IR_TREE` snapshot.

Listing 6.1: IR_TREE after dead-code cleanup

```
fn test() -> unit
    BLOCK TYPE=unit
    |   FUNCTION_CALL("external") TYPE=unit
    |   RETURN TYPE=!
    |   |   CONST UNIT TYPE=unit
```

6.10 IR_TREE Example

The `comptime_primes.epl` example computes a table of twenty primes at compile time. After compile-time evaluation, the tree no longer contains the loop that computed the values. It contains a store of one constant array:

Listing 6.2: Compile-time result in IR_TREE

```
STORE TYPE=unit
|   VARIABLE(var_1) [PLACE] TYPE=[i32; 20]
|   CONST [3, 5, 7, 11, 13, 17, 19, 23, 29, 31,
|          37, 41, 43, 47, 53, 59, 61, 67, 71, 73]
|          TYPE=[i32; 20]
```

This is the main reason `IR_TREE` is useful as both a semantic representation and a testing artifact. It shows the result of lowering and compile-time execution directly.

Chapter 7

Compile-Time Evaluation

Compile-time evaluation is the most distinctive feature of EPL. It allows a programmer to write ordinary EPL expressions that the compiler executes while compiling the program. The result is inserted back into the program as a constant. This chapter explains why the feature exists, how purity is enforced, how the evaluator represents values and memory, and how pure functions are called during compilation.

7.1 Purpose

Without compile-time evaluation, a programmer who wants a precomputed table must either write the table by hand, generate it with an external script, or compute it at run time. Each option has a drawback. Hand-written tables are error-prone. External generators add another tool and another language to the build process. Run-time computation repeats work that could have been done once during compilation.

`comptime` provides another option. The programmer writes a normal expression. The compiler type checks it, proves that it is valid for compile-

time execution, interprets it, and replaces it with a constant. The example in Listing 7.1 computes the first twenty odd prime numbers during compilation and then prints them at run time.

Listing 7.1: Compile-time prime table example

```
fn main() -> i32 {
  let primes = comptime {
    let lut: [i32; 20];
    let ready = 0;
    let next = 3;
    while ready < 20 {
      while !is_prime(next) {
        next += 2;
      }
      lut[ready as u64] = next;
      ready += 1;
      next += 2;
    }
    lut
  };
  for i in 0u64..20 {
    printf("%i ", primes[i]);
  }
  printf("\n");
  0
}
```

The important point is that the loop in the `comptime` block does not remain in the generated run-time program. It is executed by the compiler, and the result is a constant array in `IR_TREE`.

7.2 Purity Boundary

Executing arbitrary program code inside the compiler would be unsafe and non-deterministic. A compile-time expression should not read run-time variables, follow raw pointers, call external functions, or depend on the state of the future process. EPL therefore defines a purity boundary.

A `comptime` expression may use:

- numeric and boolean literals;
- arithmetic, comparison, boolean operations, and integer casts;
- blocks, conditionals, and loops;
- local variables declared inside the compile-time expression;
- assignment to those local variables;
- arrays and structs;
- `break` and `continue` for loops inside the expression;
- calls to functions annotated with `@pure`.

It may not use:

- string literals;
- surrounding run-time variables or function arguments;
- pointer address-of or dereference operations;
- impure function calls;
- `return` from the enclosing function;
- loops or variables outside the compile-time expression's context.

This purity boundary is enforced after lowering to `IR_TREE`. The checker walks the expression tree and validates each operation. Checking `IR_TREE` instead of the AST is important because the tree has resolved IDs and types. The checker can distinguish a variable declared inside the compile-time expression from a variable in an enclosing run-time scope.

7.3 Pure Functions

The `@pure` annotation marks a function that is valid to call from compile-time code. A pure function must have a body. It may use its own arguments, local variables, local mutation, ordinary control flow, and calls to other pure functions. It may not call impure functions, use strings, take addresses, or dereference pointers.

Listing 7.2 shows a pure function from the examples.

Listing 7.2: Pure function used by comptime

```
@pure
fn is_prime(x: i32) -> bool {
  if x <= 1 {
    return false;
  }
  if x == 2 {
    return true;
  }
  if x % 2 == 0 {
    return false;
  }
  let i = 3;
  while i <= x / 2 {
    if x % i == 0 {
      return false;
    }
    i += 2;
  }
  true
}
```

This function uses local mutation and early returns, but it does not depend on run-time state. Therefore the compile-time evaluator can call it with constant arguments.

7.4 Evaluator Structure

The evaluator is implemented in `src/ir_tree/evaluator.rs`. It evaluates `IR_TREE` expressions and returns an `ir_tree::Constant`. The public entry point for compile-time expressions is `eval_comptime_expr`. Pure function calls are handled by a separate internal function, `eval_pure_function`.

The evaluator uses a recursive expression interpreter. It pattern matches on `ExprKind`. Constants evaluate to themselves. Loads read from compile-time memory. Stores write to compile-time memory and return unit. Arithmetic and comparison call helper routines. Conditionals evaluate the condition and then one branch. Loops repeatedly evaluate the body until a matching break is encountered. Function calls evaluate their arguments and then interpret the body of the pure callee.

7.5 Control-Flow Signals

The evaluator needs to support `return`, `break`, and `continue`. These are not ordinary values because they change control flow. The implementation represents them as internal evaluation errors: `Return(Constant)`, `Break(LoopId, Constant)`, and `Continue(LoopId)`.

This design is similar to using exceptions internally in an interpreter. When a loop body produces a `Break` with the loop's own ID, the loop consumes the signal and produces the break value. When it produces a `Continue` with the loop's own ID, the loop starts the next iteration. If the signal belongs to an outer loop or a return, it propagates outward.

At the top level of a `comptime` expression, `return`, out-of-context `break`, and out-of-context `continue` should be impossible because the purity checker

rejects them. If one appears there, it is a compiler bug.

7.6 Constant Memory

Local mutation inside `comptime` requires memory. The evaluator cannot only store a map from variables to high-level constants because field and array element assignment require partial updates. For example:

Listing 7.3: Partial compile-time mutation

```
let lut: [i32; 20];
lut[ready as u64] = next;
```

The evaluator represents each local variable as a byte buffer sized according to the same type layout used by the compiler. A compile-time place is represented as a variable ID plus a byte offset. Loading converts bytes back into a constant. Storing converts a constant into bytes and writes them into the buffer.

The helper module `const_abi.rs` performs conversion between constants and byte buffers. It handles integers, booleans, arrays, structs, unit, and undefined values. For structs, it respects field offsets and padding. For arrays, it serializes elements in order using the element layout.

This design keeps compile-time mutation consistent with the run-time layout used later by lower IR and LLVM generation. The byte-addressed model was also chosen so that pointer support could be added to compile-time evaluation later without replacing the evaluator's storage abstraction. Such support would be restricted to pointers into local compile-time variables, with checked pointer arithmetic, explicit pointer casts, and dereferences validated against the known local memory ranges.

7.7 Arithmetic and Casts

The evaluator's arithmetic helper supports all integer types. It uses wrapping operations for addition, subtraction, multiplication, division, and remainder. Comparison supports booleans and integers. Cast evaluation supports integer-to-integer casts by using Rust casts to model truncation and extension.

Pointer constants are not supported by compile-time evaluation. This is an intentional restriction. A pointer value would either point into compile-time memory, which has no valid run-time address, or into run-time memory, which does not exist yet. Rejecting pointer operations keeps the compile-time value model simple and deterministic.

7.8 Case Study: Struct Computation

The struct example demonstrates that compile-time evaluation is not limited to primitive integers. The program defines a `Vec2` struct and a pure `vec2_add` function. At run time it prints ordinary struct values. At compile time it also evaluates:

Listing 7.4: Compile-time struct expression

```
vec2_print(comptime vec2_add(.{ x: 1, y: 2 }, .{ y: 10, x: 20 }));
```

The `IR_TREE` snapshot shows that this call becomes a constant struct:

Listing 7.5: Struct constant after compile-time evaluation

```
FUNCTION_CALL("vec2_print") TYPE=unit
|   CONST {21, 12} TYPE=StructId(0)
```

The field order in the initializer does not matter. Semantic analysis maps fields by name, and constant construction stores them in declaration

order.

7.9 Benefits and Tradeoffs

The benefit of this design is that compile-time evaluation reuses the normal language. There is no separate macro language, no string-based code generation, and no second parser. The compiler can type check code once, then either lower it to run time or interpret it during compilation.

The tradeoff is that purity rules must be conservative. Some operations that could theoretically be made safe are rejected. For example, the evaluator could eventually support references into compile-time-only memory, but the current pointer model does not distinguish compile-time pointers from run-time pointers. Rejecting pointer operations is simpler and safer for this implementation.

The current evaluator is also not optimized for speed. It repeatedly traverses the module to find and replace compile-time expressions. The documentation notes that this is inefficient. For the project examples it is sufficient, but a larger compiler would use a more direct worklist or integrate evaluation into semantic lowering.

Chapter 8

Lower IR and Control Flow

After semantic analysis and compile-time evaluation, the compiler lowers `IR_TREE` into a lower intermediate representation. This lower IR is closer to machine code and LLVM. It is built around functions, basic blocks, instructions, terminators, values, definitions, and allocation slots. This chapter explains the representation and the lowering process.

8.1 Purpose of Lower IR

`IR_TREE` is convenient for semantic analysis because it keeps expression structure. LLVM generation, however, needs explicit control flow. It must know which basic block branches to which successor, where values merge, where returns happen, and which operations produce reusable values. Lower IR provides this explicit structure.

The lower IR is not intended to be a full optimizer's IR. It does not implement dominance analysis, liveness analysis, register allocation, or source-variable SSA conversion. It is a pragmatic bridge between typed `IR_TREE` and LLVM IR. It makes enough structure explicit that LLVM

generation can be systematic.

8.2 Functions and Basic Blocks

An IR function contains a mangled name, argument types, a variadic flag, a return type, and optionally a body. A body contains a list of allocation slots, an entry basic block ID, and a map of basic block IDs to blocks. A basic block contains block arguments, instructions, and a terminator.

Block arguments are important. When a conditional or loop produces a value, the continuation block receives that value as a block argument. During LLVM generation, block arguments become phi nodes. This avoids carrying explicit phi instructions in lower IR while still modeling value merging.

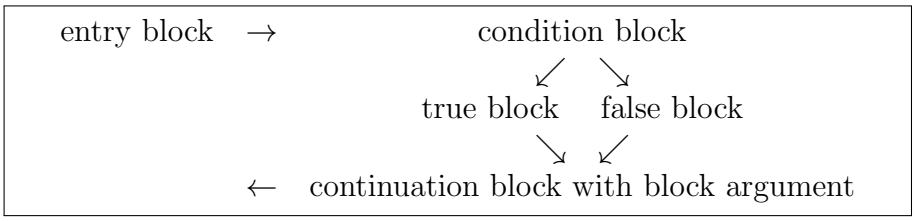

Figure 8.1: Value merge represented by a continuation block argument

8.3 Values and Definitions

Lower IR values can be zero-sized values, undefined values, booleans, strings, numbers, or definitions. A `DefinitionId` represents the result of an instruction or a block argument. Each definition has a type.

This gives the IR an SSA-like property for computed values: an instruction produces a fresh definition, and later instructions refer to it. Source variables are different. Mutable locals are represented as allocation slots, so

assignments become stores and reads become loads. This hybrid design is simple and maps naturally to LLVM `alloca`, `load`, and `store`.

8.4 Instructions

The lower IR instruction set is small:

- `Load` reads from a pointer value.
- `Store` writes a value to a pointer value.
- `FunctionCall` calls a named function.
- `Cmp` compares integer or boolean values.
- `Arithmetic` performs integer arithmetic.
- `Not` negates a boolean.
- `OffsetPtr` computes a byte offset from a pointer.
- `Zext`, `Sext`, and `Truncate` implement integer casts.

Terminator instructions are separate from ordinary instructions. A block ends in a jump, conditional jump, return, or unreachable terminator. This separation prevents accidentally appending instructions after control flow has already left the block.

8.5 Type Lowering

The lower IR type system is simpler than the `IR_TREE` type system. The `IR_TREE` distinguishes signed and unsigned integers. Lower IR collapses `i8`/`u8` into `I8`, `i32`/`u32` into `I32`, and `i64`/`u64` into `I64`. Signedness is not stored in the type. Instead, the lowering stage attaches a `signed` boolean to arithmetic, comparison, and extension instructions where signedness matters.

Pointers become a single lower `Ptr` type. Structs become lower struct types with field types and layout. Arrays become lower array types. The never type is lowered to unit because a never expression does not produce a normal value in lower IR.

8.6 Lowering Blocks and Locals

When lowering a `IR_TREE` block, the compiler creates an allocation slot for each variable declared in the block. It stores the mapping from `VariableId` to allocation definition in `variable_map`. Then it evaluates the block's expressions in order. If an expression returns `Never`, the block returns `Never` to its caller, because following expressions are unreachable.

This model is simple but not fully optimized. Every local variable becomes a stack slot even if it could be represented as an SSA value. LLVM can later optimize many such allocas in production optimization pipelines, but this compiler currently emits unoptimized object code. The choice is appropriate for the project because it keeps assignment, address-of, field updates, and array updates uniform.

8.7 Lowering Places

Places lower to pointers. A variable place lowers to the allocation slot for that variable. A dereference place lowers by evaluating the pointer expression. A field place lowers by computing the field offset and emitting an `OffsetPtr`. An array element place lowers by evaluating the array pointer and index, multiplying the index by the element size, and emitting `OffsetPtr`.

This is one of the reasons the place/value distinction in `IR_TREE` is useful. By the time lower IR generation runs, it can ask for a place as a pointer and generate load or store instructions around it.

8.8 Lowering Conditionals

When lowering an `If` expression, the compiler creates three blocks: the true block, the false block, and the continuation block. It evaluates the condition in the current block and emits a conditional jump to the true and false blocks. Each branch is then lowered independently. If a branch produces a value, it jumps to the continuation block and passes the value as an argument. If a branch never returns, it does not jump to the continuation. The continuation block receives a fresh block argument representing the value of the whole `If` expression.

This works for both ordinary conditionals and lowered short-circuit boolean operators. Because `a || b` has already become an `If`, lower IR does not need a separate short-circuit instruction.

8.9 Lowering Loops

Loop lowering creates a body block and a continuation block. The current block jumps to the body block. `continue` jumps to the body block. `break` jumps to the continuation block and passes the break value as an argument. If the loop body reaches the end normally, the compiler emits a jump back to the body block.

The lowering context stores maps from `LoopId` to break and continue targets. This is why `LoopId` is assigned during `IR_TREE` lowering. Nested

loops work because each `break` and `continue` carries the specific loop ID it targets.

8.10 Aggregates

Struct and array values require special handling. A simple scalar expression can be evaluated to a lower IR value directly. An aggregate initializer may require writing several fields or elements into memory. The lowering context therefore has helper methods that evaluate array initializers and struct initializers into a provided pointer.

For a struct initializer, the compiler evaluates field expressions, computes each field offset from the semantic type layout, and stores each value into the appropriate offset. For an array initializer, it evaluates all element expressions, computes each element offset from the element layout, and stores the values in order. Constants for aggregate values are lowered similarly by allocating storage, writing the constant into that storage, and loading the aggregate value.

8.11 Cleanup Passes

After lowering, `ir::opt::basic_passes` runs two passes: `drop_zst` and `simplify_cfg`.

`drop_zst` removes operations on zero-sized types. It removes zero-sized function arguments, block arguments, allocation slots, loads, stores, and normalizes zero-sized returns. This matters because `unit` is represented as a type in the compiler but should not create useless run-time memory operations.

`simplify_cfg` removes unreachable blocks, merges a block with its only successor when the successor has a single predecessor, and redirects empty jump blocks to their targets. It also maintains a rename map so references to merged block arguments are updated correctly.

These passes are small, but they improve the readability of lower IR and make LLVM generation simpler. They also demonstrate how compiler optimizations can start with local transformations before moving to more advanced analyses.

8.12 Graphviz Output

The `cfg` command renders lower IR as a Graphviz DOT graph. Each function becomes a subgraph. Each basic block is shown with its arguments, instructions, and terminator information. Edges show jumps and conditional branches. This is a debugging feature, but it is also useful for learning. Many compiler concepts are easier to understand when control flow is visible as a graph instead of only as a linear debug dump.

The following two examples illustrate how source language constructs map to basic blocks in the generated CFG.

Listing 8.1 shows a function that selects between two values using a conditional expression. The generated CFG in Figure 8.2 reveals the four-block structure produced by conditional lowering: an entry block that evaluates `flag` and emits a conditional jump, a true block, a false block, and a continuation block that receives the selected value as a block argument.

Listing 8.1: select.epl: conditional expression

```
fn select(a: i32, b: i32, flag: bool) -> i32 {
    if flag { a } else { b }
}
```

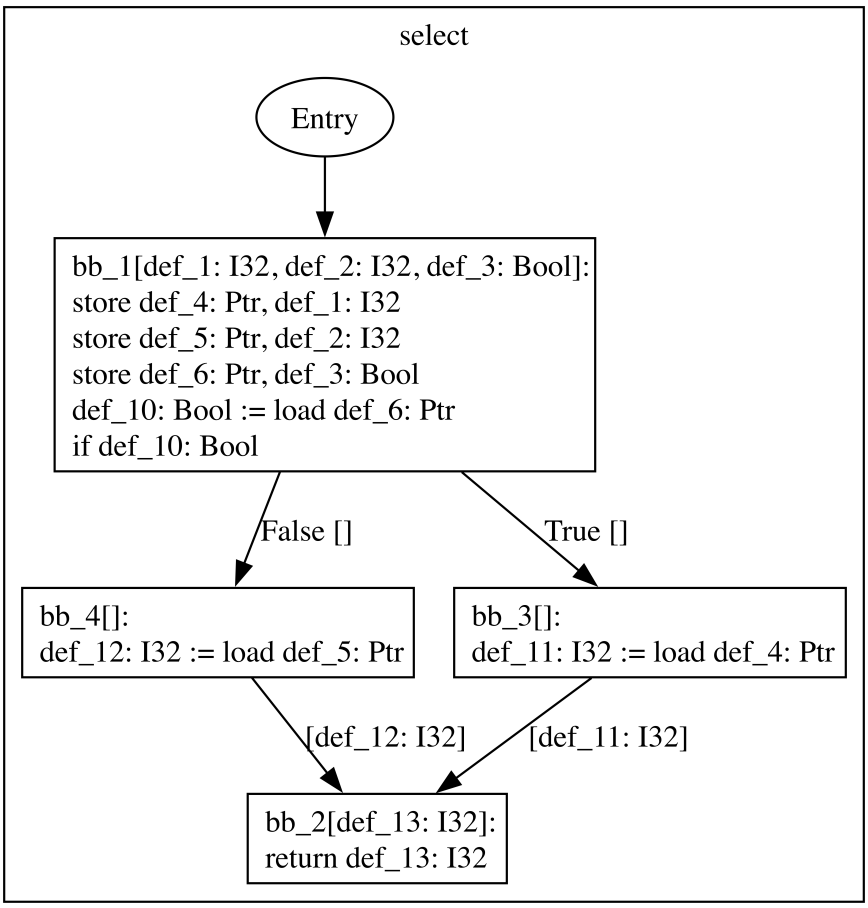

Figure 8.2: CFG for `select`: entry block branches to the true and false blocks, which both pass their result to a shared continuation block argument

Listing 8.2 shows a function that accumulates a sum over a range using a `while` loop. The generated CFG in Figure 8.3 shows the loop structure: an entry block that falls through to the body block, a body block that performs the loop work and loops back to itself via a conditional jump, and a continuation block reached when the loop condition is false.

Listing 8.2: sum_range.epl: while loop accumulation

```
fn sum_range(from: i32, to: i32) -> i32 {
```

```
    let sum = 0;
    while from < to {
        sum += from;
        from += 1;
    }
    sum
}
```

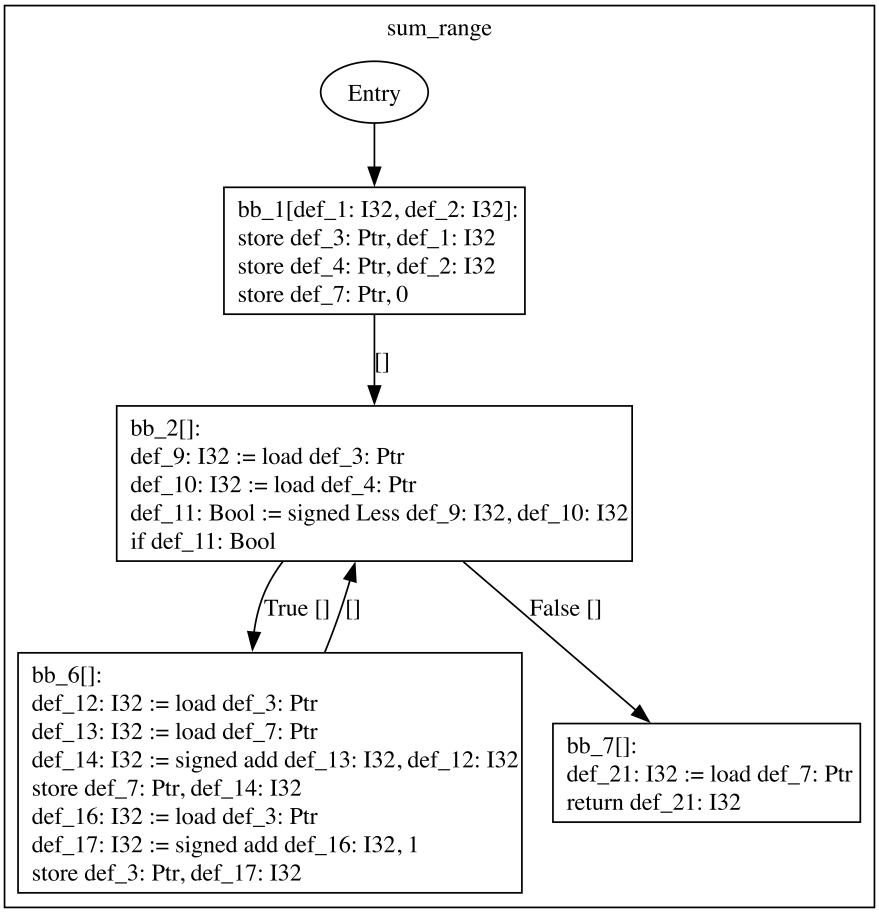

Figure 8.3: CFG for `sum_range`: the body block loops back to itself while the condition holds, and exits to the continuation block when the condition fails

8.13 Tradeoffs

The lower IR is intentionally pragmatic. It is not a research-grade SSA optimizer. It does not eliminate all loads and stores, it does not perform constant propagation, it does not inline functions, and it does not select machine instructions. Instead, it provides exactly the structure needed to bridge `IR_TREE` and LLVM.

This is the right tradeoff for EPL because LLVM already provides a strong backend. The project value is in showing the full path from source language design to LLVM generation, not in replacing LLVM's optimizer or target infrastructure.

Chapter 9

LLVM Backend and Linking

The final compiler stage maps lower IR to LLVM IR and asks LLVM to emit a native object file. This chapter explains the LLVM backend implementation, including function declaration, type mapping, value mapping, basic block construction, phi node construction, object emission, and linking.

9.1 Why LLVM

Writing a complete native backend is a large project. A backend must understand instruction selection, register allocation, stack layout, object file formats, relocations, calling conventions, debug information, and target-specific details. LLVM provides reusable infrastructure for many of these tasks [7, 8]. By targeting LLVM IR, EPL can focus on source-language design and compiler lowering while still producing native object files.

The project originally considered multiple possible backends in its proposal, including LLVM and self-contained minimal backends. The implemented compiler uses LLVM because it gives the project an end-to-end native-code path within the time available for a final year project.

9.2 LLVM Module Construction

The LLVM backend is implemented in `src/llvm.rs`. The main public function is `LlvmModule::from_ir`. It takes lower IR and returns an LLVM module wrapper. The wrapper owns an LLVM context and LLVM module pointer and disposes of them when dropped.

LLVM construction happens in two phases. First, the compiler declares every function and records its LLVM function type and LLVM function value. This allows calls to functions defined later in the file. Second, the compiler emits the body of each function that has a body. External declarations are left as LLVM declarations.

This two-phase structure mirrors semantic analysis. Function signatures are known before bodies are emitted. This is necessary for recursion and forward calls.

9.3 Type Mapping

The LLVM type mapping is direct:

Lower IR type	LLVM type
`Unit`	empty LLVM struct
`Bool`	`i1`
`I8`	`i8`
`I32`	`i32`
`I64`	`i64`
`Ptr`	opaque LLVM pointer
`Struct`	LLVM struct with lowered field types
`Array`	LLVM array with lowered element type and length

Table 9.1: Mapping from lower IR types to LLVM types

LLVM uses a distinct `i1` type for booleans. Integer widths map directly. Pointers use LLVM opaque pointers. Unit values become an empty struct, although lower IR cleanup removes most zero-sized operations before LLVM generation.

9.4 Values

The backend maintains a map from lower IR definitions to LLVM values. Function arguments are inserted into the map from the LLVM function parameters. Allocation slots are emitted in the entry block using LLVM `alloca` and inserted into the map. Instruction results are inserted as instructions are emitted. Block arguments are inserted as LLVM phi nodes before block instructions are emitted.

Constants are translated as needed. Boolean constants become LLVM integer constants of type `i1`. Number constants become LLVM integer constants of the correct width. Undefined values become LLVM `undef`. String constants are emitted as global arrays initialized with the string data and null terminator.

9.5 Basic Blocks and Phi Nodes

For each lower IR basic block, the backend creates an LLVM basic block. It then creates phi nodes for non-entry block arguments. The actual incoming values are filled when predecessor terminators are emitted.

This maps lower IR block arguments to LLVM's phi-node representation. For a jump terminator, the backend emits an LLVM branch and then adds incoming values to the target block's phi nodes. For a conditional jump, it

emits an LLVM conditional branch and adds incoming values to the phi nodes of both successors.

This design keeps lower IR independent from LLVM's exact phi instruction API while still preserving SSA-style value merging.

9.6 Instruction Emission

Each lower IR instruction maps to one or more LLVM builder calls:

- `Load` maps to `LLVMBuildLoad2`;
- `Store` maps to `LLVMBuildStore`;
- `FunctionCall` maps to `LLVMBuildCall2`;
- `Cmp` maps to `LLVMBuildICmp` with signed or unsigned predicates;
- `Arithmetic` maps to add, sub, mul, signed/unsigned division, or signed/unsigned remainder;
- `Not` maps to XOR with `true`;
- `OffsetPtr` maps to `LLVMBuildGEP2` over `i8`;
- `Zext`, `Sext`, and `Truncate` map to LLVM integer cast instructions.

Signedness is handled at the instruction level. For example, a lower IR comparison stores whether the operands should be interpreted as signed. The LLVM backend chooses `LLVMIntSLT` or `LLVMIntULT` for less-than based on that flag.

9.7 Terminator Emission

Lower IR terminators map directly to LLVM terminators. A jump becomes an LLVM branch. A conditional jump becomes an LLVM conditional branch.

A return becomes an LLVM return with a value. An unreachable terminator becomes LLVM `unreachable`.

The backend must ensure that each LLVM basic block ends with exactly one terminator. This is one reason lower IR separates instructions from terminators. The structure prevents accidental emission of non-terminator instructions after a branch or return.

9.8 Module Verification

After constructing the LLVM module, the compiler calls `LLVMVerifyModule`. If verification fails, the compiler prints the LLVM verifier error and exits. Verification is important because LLVM IR has structural rules that are easy to violate when building IR through an API. Examples include missing terminators, phi nodes with incorrect incoming blocks, mismatched call signatures, or invalid types.

LLVM verification does not prove that the original EPL program is safe or that the generated code is semantically correct. It only checks LLVM IR well-formedness. It is nevertheless an important backend safety net.

9.9 Object File Emission

The `llvm-obj` command calls `LlvmModule::compile`. The compile method initializes LLVM targets, obtains the default target triple, creates a target machine, sets the module data layout and target triple, and asks LLVM to emit an object file named `a.out.o`.

The compiler currently emits object files with no LLVM optimization pipeline enabled. There is commented code showing where a pass pipeline

could be run in the future. This is a reasonable current decision because the project focuses on correct lowering rather than performance optimization.

9.10 Linking

The compiler stops at an object file. To produce an executable, the object file is linked with a system C compiler:

Listing 9.1: Compiling and linking an EPL program

```
cargo run -- llvm-obj examples/hello_world.epl
cc a.out.o
./a.out
```

This link step resolves external declarations such as `printf` and `exit`. It also lets the platform C toolchain handle startup files and libc linkage. A future compiler could invoke the linker automatically, but keeping the link step explicit makes the current architecture simpler.

9.11 FFI Limits

Although EPL can call simple external C functions, it should not be described as having a complete C FFI. The implemented path works for examples such as `printf(fmt: *i8, ...)` and `exit(code: i32) -> !`. It does not fully model all C ABI cases.

Important missing cases include by-value aggregate arguments and returns, platform-specific varargs details, exact C `void` mapping for every signature shape, structure packing pragmas, integer promotion rules for varargs, callbacks, and library header parsing. These features are not needed to demonstrate the compiler pipeline, but they are important for future work if EPL is to call arbitrary C libraries reliably.

9.12 Backend Tradeoffs

The LLVM backend is direct and readable. It does not attempt to hide LLVM behind a large abstraction layer. The advantage is that every lower IR construct has an obvious LLVM emission point. The disadvantage is that target-specific details are partly delegated to LLVM without building a higher-level ABI model inside EPL.

For a final year compiler project, this is an appropriate tradeoff. It produces native object code, gives access to LLVM verification, and keeps the backend small enough to reason about.

Chapter 10

Testing, Evaluation, and Future Work

This chapter evaluates the current implementation. The evaluation focuses on correctness of the implemented compiler pipeline, example coverage, snapshot testing, and the limitations that remain. It also presents future work that would move EPL closer to a production language.

10.1 Testing Strategy

The project uses a lightweight test harness in the `epl_test` crate. The harness recursively scans provided directories for `.epl` files. For each source file, it performs two checks.

First, it runs the compiler in `ir_tree` mode and compares the output to a neighboring `.epl.ir_tree` snapshot. If the snapshot is missing, the harness writes a new one and marks the test as new. If the snapshot exists but differs, the test fails. This catches changes to semantic lowering, type checking, compile-time evaluation, and tree cleanup.

Second, it runs the compiler in `ir` mode as a smoke test. The current harness does not snapshot lower IR output, but it verifies that lower IR generation succeeds for every example and regression program.

10.2 Verification Result

The implementation was built and tested in the project workspace. The compiler build command completed successfully:

Listing 10.1: Compiler build command

```
cargo build
```

The build produced one platform linker warning about a macOS library deployment target, but the compiler binary was produced successfully. The test harness was then run with:

Listing 10.2: Test harness command

```
cargo run -p epl_test ./target/debug/epl examples tests
```

The result was:

Listing 10.3: Test harness summary

```
DONE (24 passed, 0 failed, 0 new)
```

The 24 checks come from 12 `.epl` source files. Each file is tested once against its `IR_TREE` snapshot and once as a lower IR smoke test.

10.3 Example Coverage

The example and test programs cover the main implemented language features.

Table 10.1: Example and regression source files

Source file	Feature coverage
`examples/hello_world.epl`	External variadic declaration and string literal call to `printf`.
`examples/fibonacci.epl`	Recursive function, iterative function, `while`, `for`, arithmetic, and external printing.
`examples/fibonacci_array.epl`	Fixed-size arrays, array mutation, array return, and array printing.
`examples/comptime_primes.epl`	`comptime`, pure function calls, nested loops, arrays, casts, and compile-time mutation.
`examples/struct.epl`	Struct definition, named and unnamed initializers, by-value struct calls, field access, and compile-time struct computation.
`examples/pointers.epl`	Address-of, typed pointer dereference, and pointer mutation.
`examples/elseif.epl`	Nested `else if` expressions and branch result typing.
`examples/exit.epl`	External never-returning function declaration.
`tests/dead_elimination.epl`	Tree cleanup after `return`.
`tests/useless_nested_blocks.epl`	Block simplification.
`tests/redundant_deref_ops.epl`	Simplification of redundant address/dereference pairs.
`tests/i32_min.epl`	Integer literal edge case.

This coverage is useful, but it is not exhaustive. It tests representative successful programs and a few optimization cases. It does not yet include many negative tests for diagnostics, ABI edge cases, malformed programs, or platform-specific behavior.

10.4 Snapshot Testing

Snapshot testing is particularly effective for this compiler because `IR_TREE` is readable. When a lowering rule changes, the generated snapshot changes in a way that can be inspected by the developer. For example, the `redundant_deref_ops.epl` test verifies that `(&a).*` becomes a load from `a` and that `&(a_ptr.*)` becomes a load of `a_ptr`. The snapshot exposes the

optimizer's behavior directly.

Snapshot testing also has risks. A developer can accidentally accept an incorrect new snapshot. The snapshot format is not a formal language specification. It is a debugging representation. For this project, however, the benefits are significant because the snapshots document the compiler pipeline and catch regressions cheaply.

10.5 Manual Inspection

The command-line modes support manual inspection beyond the automated harness. For example:

Listing 10.4: Inspecting generated LLVM IR

```
cargo run -- llvm-ir examples/hello_world.epl
```

The `cfg` command can be combined with Graphviz:

Listing 10.5: Generating a CFG graph

```
cargo run -- cfg examples/fibonacci.epl > /tmp/cfg.dot
dot -Tsvg /tmp/cfg.dot -o /tmp/cfg.svg
```

These inspection modes are valuable during compiler development because they make each stage visible. They are also useful for explaining the project during presentation or assessment.

10.6 Correctness Discussion

The current tests show that the implemented examples lower successfully and that `IR_TREE` output remains stable. They do not prove full compiler correctness. Compiler correctness would require a much broader strategy:

negative tests, randomized programs, differential testing against a reference interpreter, run-time output checks, LLVM IR checks, and tests on multiple platforms.

Even so, the current verification is meaningful. It exercises the full front end and semantic pipeline for every example. It exercises compile-time evaluation through the prime and struct examples. It exercises lower IR generation through the smoke tests. It verifies that the build environment can compile the Rust compiler.

10.7 Performance

The thesis does not present a detailed performance benchmark. The compiler is not optimized for compile-time speed or generated-code speed. The most important performance-related facts are architectural:

- compile-time evaluation currently searches and replaces `comptime` expressions in a simple repeated traversal;
- lower IR represents local variables as stack allocation slots instead of promoting them to SSA values;
- LLVM object emission is performed without an enabled optimization pass pipeline;
- tree and CFG cleanup are intentionally local and conservative.

This is acceptable for the project scope. The compiler is designed to be understandable and complete, not fast. Future work can add performance benchmarks once language semantics and test coverage are stronger.

10.8 Current Limitations

The main limitations of EPL are:

- one source file is compiled at a time;
- there are no modules, imports, packages, or separate compilation;
- arrays require literal lengths in type expressions;
- there are no generics;
- there is no standard library;
- pointer operations are unsafe and unchecked;
- array indexing has no bounds checks;
- the C ABI is only partially modeled;
- diagnostics stop at the first error and do not recover;
- lower IR optimizations are minimal;
- compile-time evaluation cannot use pointers or strings;
- there is no automatic final link command.

These limitations are expected for a final year compiler project. The important point is that they are explicit. The implemented compiler has a coherent subset rather than a large number of partially designed features.

10.9 Future Work

The first future improvement is stronger testing. The project should add negative tests for parser errors, type errors, purity violations, invalid casts, invalid field accesses, invalid array indexing, bad `main` signatures, and malformed function declarations. The harness could also compare expected diagnostic output.

The second improvement is run-time output testing. The compiler already emits object files. A test runner could compile selected examples, link them in a temporary directory, run the executables, and compare stdout

and exit status. This would test the LLVM backend and FFI path more directly than lower IR smoke tests.

The third improvement is modules. Even a simple import system would require decisions about file discovery, duplicate definitions, visibility, cyclic dependencies, and separate or whole-program compilation. Modules would make the language more usable but would significantly expand the compiler architecture.

The fourth improvement is a more complete ABI model. Reliable C interop requires platform-specific rules for aggregate passing, varargs, return values, alignment, and symbol naming. This would make EPL more practical as a systems language.

The fifth improvement is safety analysis. Bounds checks could be added for arrays, and pointer operations could be made more explicit or restricted. Implementing ownership or borrow checking would be a much larger research direction, but smaller safety steps are possible.

The sixth improvement is optimization. Local alloca promotion, constant propagation, dead-store elimination, common subexpression elimination, and LLVM optimization pipelines could improve output quality. The current lower IR is a good starting point because it already has basic blocks and immutable definition IDs.

The seventh improvement is richer compile-time evaluation. It could support constant expressions in array lengths, more built-in compile-time functions, compile-time strings, or a separate model for compile-time-only references. Each addition should preserve the purity boundary so that compile-time execution remains deterministic.

10.10 Conclusion

This thesis presented the design and implementation of EPL, a small ahead-of-time compiled language with compile-time evaluation. The compiler implements a complete path from source text to native object code: lexing, parsing, semantic analysis, typed `IR_TREE` construction, purity checking, compile-time interpretation, lower CFG IR generation, cleanup passes, LLVM IR generation, and object emission.

The project demonstrates that a compact compiler can still include modern language ideas such as expression-oriented control flow and compile-time execution. The implementation is intentionally conservative, but it is complete enough to compile examples involving recursion, loops, arrays, structs, pointers, external calls, and non-trivial compile-time computation. The current test harness verifies the examples and regression tests with 24 passing checks.

The result is an educational compiler and language specification. It is not a production language, but it is a solid foundation for further work in modules, diagnostics, ABI support, safety, optimization, and richer compile-time metaprogramming.

Appendix A

EPL Syntax Reference

This appendix records the implemented syntax in compact EBNF form. The grammar is based on the syntax reference included with the project.

A.1 Notation

Listing A.1: Grammar notation

```
::=      defines a production
|        separates alternatives
{ ... }  repeats zero or more times
[ ... ]  marks an optional part
'...'    marks a literal token
```

A.2 Source and Items

Listing A.2: Top-level grammar

```
source ::= { item }
ident  ::= /[a-zA-Z_][a-zA-Z_0-9]*/ except keyword

item ::= { annotation } ( function | struct_def )

annotation ::= '@' ident
```

A.3 Functions and Structs

Listing A.3: Function and struct grammar

```
function ::= 'fn' ident '(' [ fn_args ] ')' [ '->' type ]
             ( block_expr | ';' )

fn_args  ::= '...' | fn_arg { ',' fn_arg } [ ',' [ '...' ] ]
fn_arg   ::= ident ':' type

struct_def    ::= 'struct' ident '{' struct_fields '}'
struct_fields ::= [ struct_field { ',' struct_field } [ ',' ] ]
struct_field  ::= ident ':' type
```

A.4 Expressions

Listing A.4: Expression grammar

```
expr ::= 'return' [ expr ]
       | 'break' [ expr ]
       | 'continue'
       | 'comptime' expr
       | assigning_expr

assigning_expr       ::= or_expr [ ( '=' | '+=' | '-=' | '*=' |
   '/=' | '%=' ) or_expr ]
or_expr              ::= and_expr { '||' and_expr }
and_expr             ::= comp_expr { '&&' comp_expr }
comp_expr            ::= range_expr [ ( '==' | '!=' | '<=' | '>='
   | '<' | '>' ) range_expr ]
range_expr           ::= additive_expr [ '..' additive_expr ]
additive_expr        ::= multiplicative_expr { ( '+' | '-' )
   multiplicative_expr }
multiplicative_expr  ::= as_expr { ( '*' | '/' | '%' ) as_expr }
as_expr              ::= unary_expr { 'as' type }
unary_expr           ::= ( '-' | '!' | '&' ) unary_expr |
   field_access_expr
field_access_expr    ::= base_expr { ( '.' '*' | '.' ident | '['
   expr ']' ) }
```

Listing A.5: Base expressions and statements

```
base_expr            ::= literal
                       | function_call_expr
                       | ident
                       | '(' expr ')'
                       | expr_with_block
                       | array_initializer

expr_with_block      ::= block_expr | if_expr | loop_expr |
   while_expr
                       | for_expr | struct_initializer

array_initializer    ::= '[' [ expr { ',' expr } [ ',' ] ] ']'

struct_initializer   ::= [ ident ] '.{' struct_initializer_fields
   '}'
struct_initializer_fields ::= [ struct_initializer_field
                              { ',' struct_initializer_field } [
    ',' ] ]
struct_initializer_field  ::= ident ':' expr

statement            ::= ';' | let_statement | expr_with_no_block
   ';'
                       | expr_with_block
block_expr           ::= '{' { statement } [ expr ] '}'
if_expr              ::= 'if' expr block_expr [ 'else' (
   block_expr | if_expr ) ]
loop_expr            ::= 'loop' block_expr
while_expr           ::= 'while' expr block_expr
for_expr             ::= 'for' ident 'in' expr block_expr
function_call_expr   ::= ident '(' [ expr { ',' expr } [ ',' ] ]
   ')'
let_statement        ::= 'let' ident ':' type ';'
                       | 'let' ident [ ':' type ] '=' expr ';'
```

A.5 Types and Literals

Listing A.6: Type and literal grammar

```
type ::= '!' | ident | '*' type | '[' type ';' expr ']'

literal              ::= number_literal | string_literal |
   bool_literal | 'undefined'
number_literal       ::= /[0-9]+/ [ number_literal_suffix ]
```

```
number_literal_suffix ::= ident
string_literal        ::= '"' { string_char | escape_sequence }
    '"'
string_char           ::= any non-quote, non-backslash character
escape_sequence       ::= '\n' | '\r' | '\t' | '\\'
bool_literal          ::= 'true' | 'false'
```

A.6 Keywords

Listing A.7: Reserved keywords

```
fn return break continue if else loop while for in let true false
struct undefined as comptime
```

The lexer treats everything after `#` as a line comment. There must be no whitespace between a number literal and its suffix.

Appendix B

Selected EPL Programs

This appendix includes representative programs from the repository. They are used by the test harness and by the thesis discussion.

B.1 Hello World

Listing B.1: examples/hello_world.epl

```
fn main() -> i32 {
  printf("hello world\n");
  0
}

fn printf(fmt: *i8, ...);
```

B.2 Fibonacci

Listing B.2: examples/fibonacci.epl

```
fn fib(n: u32) -> u32 {
  if n < 2 {
    n
  } else {
```

```
    fib(n - 2) + fib(n - 1)
  }
}

fn fib_it(n: u32) -> u32 {
  let a: u32 = 0;
  let b: u32 = 1;
  let i: u32 = 0;
  while i < n {
    let tmp = a;
    a = b;
    b += tmp;
    i += 1;
  }
  a
}

fn main() -> i32 {
  for n in 0u32..10 {
    printf("fib(%i) = %i\n", n, fib(n));
    printf("fib_it(%i) = %i\n", n, fib_it(n));
  }
  0
}

fn printf(fmt: *i8, ...);
```

B.3 Compile-Time Primes

Listing B.3: examples/comptime_primes.epl

```
fn main() -> i32 {
  let primes = comptime {
    let lut: [i32; 20];
    let ready = 0;
    let next = 3;
    while ready < 20 {
      while !is_prime(next) {
        next += 2;
      }
      lut[ready as u64] = next;
      ready += 1;
      next += 2;
```

```
    }
    lut
  };
  for i in 0u64..20 {
    printf("%i ", primes[i]);
  }
  printf("\n");
  0
}

@pure
fn is_prime(x: i32) -> bool {
  if x <= 1 {
    return false;
  }
  if x == 2 {
    return true;
  }
  if x % 2 == 0 {
    return false;
  }
  let i = 3;
  while i <= x / 2 {
    if x % i == 0 {
      return false;
    }
    i += 2;
  }
  true
}

fn printf(fmt: *i8, ...);
```

B.4 Conway's Game of Life

Listing B.4: examples/game_of_life.epl

```
struct Board {
  rows: [[bool; 20]; 20]
}

fn main() -> i32 {
  let board: Board;
```

```
  clear_board(&board);
  seed_board(&board);

  loop {
    print_board(&board);
    update_board(&board);
    usleep(250000);
  }
}

fn clear_board(board: *Board) {
  for row in 0u64..20 {
    for col in 0u64..20 {
      set_cell(board, row, col, false);
    }
  }
}

fn seed_board(board: *Board) {
  set_cell(board, 8, 10, true);
  set_cell(board, 9, 11, true);
  set_cell(board, 10, 9, true);
  set_cell(board, 10, 10, true);
  set_cell(board, 10, 11, true);
}

fn print_board(board: *Board) {
  printf("\n");
  for row in 0u64..20 {
    for col in 0u64..20 {
      printf(if get_cell(board, row, col) { "#" } else { "." });
    }
    printf("\n");
  }
}

fn update_board(board: *Board) {
  let next: Board;

  for row in 0u64..20 {
    for col in 0u64..20 {
      let alive = get_cell(board, row, col);
      let neighbors = get_alive_neighbors(board, row, col);
      let next_cell = if alive { neighbors == 2 || neighbors == 3
    } else { neighbors == 3 };
      set_cell(&next, row, col, next_cell);
    }
```

```
  }

  board.* = next;
}

fn get_alive_neighbors(board: *Board, row: u64, col: u64) -> u64 {
  let count = 0u64;

  for r in (row + 20 - 1)..(row + 20 + 2) {
    for c in (col + 20 - 1)..(col + 20 + 2) {
      let r = r % 20;
      let c = c % 20;
      if (r != row || c != col) && get_cell(board, r, c) {
        count += 1;
      }
    }
  }

  count
}

fn get_cell(board: *Board, row: u64, col: u64) -> bool {
  board.*.rows[row][col]
}

fn set_cell(board: *Board, row: u64, col: u64, alive: bool) {
  board.*.rows[row][col] = alive;
}

fn printf(fmt: *i8, ...);
fn usleep(usec: u32) -> i32;
```

Appendix C

Representative Generated Output

This appendix includes selected generated output used to inspect the compiler pipeline. The output is shortened where necessary for readability.

C.1 IR_TREE for Hello World

Listing C.1: Typed tree for hello world

```
fn main() -> i32
    BLOCK TYPE=i32
    |   FUNCTION_CALL("printf") TYPE=unit
    |   |   CONST_STRING("hello world\n") TYPE=*i8
    |   CONST 0 TYPE=i32

fn printf(fmt: *i8, ...) -> unit
```

C.2 IR_TREE for Redundant Pointer Operations

Listing C.2: Simplified redundant address and dereference operations

```
fn main() -> i32
    BLOCK TYPE=i32
    |   DECLARE var_1 "a" : i32
    |   DECLARE var_2 "a_ptr" : *i32
    |   DECLARE var_3 "b" : i32
    |   DECLARE var_4 "c" : *i32
    |   STORE TYPE=unit
    |   |   VARIABLE(var_1) [PLACE] TYPE=i32
    |   |   CONST 42 TYPE=i32
    |   STORE TYPE=unit
    |   |   VARIABLE(var_2) [PLACE] TYPE=*i32
    |   |   GET_POINTER TYPE=*i32
    |   |   |   VARIABLE(var_1) [PLACE] TYPE=i32
    |   STORE TYPE=unit
    |   |   VARIABLE(var_3) [PLACE] TYPE=i32
    |   |   LOAD TYPE=i32
    |   |   |   VARIABLE(var_1) [PLACE] TYPE=i32
    |   STORE TYPE=unit
    |   |   VARIABLE(var_4) [PLACE] TYPE=*i32
    |   |   LOAD TYPE=*i32
    |   |   |   VARIABLE(var_2) [PLACE] TYPE=*i32
    |   CONST 0 TYPE=i32
```

C.3 IR_TREE for Compile-Time Primes

Listing C.3: Compile-time prime table after evaluation

```
fn main() -> i32
    BLOCK TYPE=i32
    |   DECLARE var_1 "primes" : [i32; 20]
    |   STORE TYPE=unit
    |   |   VARIABLE(var_1) [PLACE] TYPE=[i32; 20]
    |   |   CONST [3, 5, 7, 11, 13, 17, 19, 23, 29, 31,
    |   |          37, 41, 43, 47, 53, 59, 61, 67, 71, 73]
    |   |          TYPE=[i32; 20]
    |   ...
```

C.4 Diagnostic Form

The diagnostic printer reports an error message, file and line, the relevant source line, and a caret underline. A representative diagnostic has this shape:

Listing C.4: Representative diagnostic shape

```
error: expected expr of type i32, found bool
at example.epl:3
    let x: i32 = true;
                 ^^^^
```

C.5 Build and Test Commands

Listing C.5: Commands used to build and test the project

```
cargo build
cargo run -p epl_test ./target/debug/epl examples tests
```

The test run in this thesis completed with:

Listing C.6: Observed test result

```
DONE (24 passed, 0 failed, 0 new)
```

C.6 Useful Inspection Commands

Listing C.7: Compiler inspection commands

```
target/debug/epl ast examples/hello_world.epl
target/debug/epl ir_tree examples/comptime_primes.epl
target/debug/epl ir examples/fibonacci.epl
target/debug/epl cfg examples/fibonacci.epl
target/debug/epl llvm-ir examples/hello_world.epl
target/debug/epl llvm-obj examples/hello_world.epl
```

The object-file command emits `a.out.o`. The executable can be produced by linking that object with a system C compiler.

Bibliography

[1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Pearson, 2 edition, 2006.

[2] Keith D. Cooper and Linda Torczon. *Engineering a Compiler*. Morgan Kaufmann, 2 edition, 2011.

[3] Robert Nystrom. Crafting interpreters. https://craftinginterpreters.com/, 2021. Accessed May 2026.

[4] Steven S. Muchnick. *Advanced Compiler Design and Implementation*. Morgan Kaufmann, 1997.

[5] Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. Efficiently computing static single assignment form and the control dependence graph. *ACM Transactions on Programming Languages and Systems*, 13(4):451–490, 1991.

[6] Neil D. Jones, Carsten K. Gomard, and Peter Sestoft. *Partial Evaluation and Automatic Program Generation*. Prentice Hall, 1993.

[7] Chris Lattner and Vikram Adve. Llvm: A compilation framework for lifelong program analysis and transformation. In *Proceedings of the International Symposium on Code Generation and Optimization*, pages 75–86, 2004.

[8] LLVM Project. Llvm language reference manual. https://llvm.org/docs/LangRef.html, 2026. Accessed May 2026.